

Final Project 3202: Playing Alien Atari with Deep Q-Learning

Introduction

For my project, I wanted to try training a DQN (Deep Q-Network) Agent to play an Atari game, using game frames and Deep Q-Learning. My main inspiration for this project was the Deepmind Technologies paper “Playing Atari with Deep Reinforcement Learning” (Mnih et al., 2013). My project borrows many ideas from this paper, the most prominent being the architecture for the CNN (Convolutional Neural Network), which is the model we will be using.

The strategy used in the paper was Deep Q-Learning with Experience Replay. This can be summarized as:

1. Maintain a “replay buffer” (essentially a container) of agent experiences after taking an action. An experience looks like (state, action, reward, nextState), where state and nextState are stacks of game frames.
2. Use an epsilon-greedy policy to choose an action (probability epsilon of sampling a random action from the action space, probability $1 - \epsilon$ of using your model to choose an action).
3. Take a step in the environment with the selected action. Store the resulting (state, action, reward, nextState) as an experience.
4. Optimize the model by sampling a fixed-size batch of experiences from the replay buffer. Apply Q-learning updates to these batches, and optimize the model.
5. Repeat for a fixed number of episodes.

Being completely new to deep learning frameworks and computer vision in general, I thought it would be helpful to use an existing DQN project and tweak it to better fit my purposes. I ended up going with PyTorch’s DQN Tutorial (Paszke and Towers, 2017). This tutorial uses the CartPole-v1 environment from Gymnasium, and does not involve image processing or feature extraction for model training. Therefore, I felt it was different enough while still being extremely helpful for managing certain aspects of the framework and project setup. The optimize function that I use in my code (alien_dqn.py) is borrowed from their tutorial, so full credit goes to the original authors.

Both these resources are linked in the references in the last section of this report.

Environment

I chose the Alien environment from Gymnasium's Atari environments. I chose this environment because I think the Alien franchise is cool, and also because the gameplay and map design are somewhat similar to the Pacman environment we worked on during class this semester.

Gameplay

The environment is a mazelike spaceship. Your goal is to crush alien eggs by running over them, while avoiding or destroying aliens that are wandering the hallways. You can destroy aliens either with your flamethrower (that has limited fuel, and does not work in the hallways to the very right and left), or with a Pulsar rifle that you can pick up (this causes the aliens to become weakened, and you can destroy them by running over them).

You can also enter a Hyperwarp Passage (on either side of the ship) that allows you to enter the other side of the ship. Alien movement patterns and whether or not the Pulsar rifle affects them depends on the game's difficulty level. You complete a level by clearing all the eggs on the ship. Rewards can be obtained by crushing eggs, picking up the Pulsar rifle, destroying aliens, and reaching the "completed" screen. You can also pick up "prizes" that appear around the map for extra rewards.

You have three humans (lives). If you complete the level you can try a bonus level, followed by a more difficult level. Points and number of remaining humans persists across completed levels.

Controls

This game was originally to be played with an Atari joystick controller and fire button.

The controls are as follows:

Action Index	Action	Controls
0	NOOP	No operation (do nothing)
1	FIRE	Press fire button without moving joystick
2	UP	Move joystick up
3	RIGHT	Move joystick right
4	LEFT	Move joystick left

5	DOWN	Move joystick down
6	UPRIGHT	Perform UP and RIGHT
7	UPLEFT	Perform UP and LEFT
8	DOWNRIGHT	Perform DOWN and RIGHT
9	DOWNLEFT	Perform DOWN and LEFT
10	UPFIRE	Perform UP and FIRE
11	RIGHTFIRE	Perform RIGHT and FIRE
12	LEFTFIRE	Perform LEFT and FIRE
13	DOWNFIRE	Perform DOWN and FIRE
14	UPRIGHTFIRE	Perform UP and RIGHT and FIRE
15	UPLEFTFIRE	Perform UP and LEFT and FIRE
16	DOWNRIGHTFIRE	Perform DOWN and RIGHT and FIRE
17	DOWNLEFTFIRE	Perform DOWN and LEFT and FIRE

Rewards

Object rewards are as follows:

Object	Reward
Crush egg	10
Pick up Pulsar rifle	100
Destroy 1st alien	500
Destroy 2nd alien	1000
Destroy 3rd alien	2000
Reach completed screen	1

Prize	Reward
Rocket	500
Saturn	1000
Star Ship	2000
1st Surprise	2000-3000
2nd Surprise	3000
3rd Surprise	5000

Setup and Model Details

Preprocessing

I used Gymnasium's Observation environment parameters and wrappers to preprocess the environment observations in a similar manner to the reference paper. I set the `obs_type` to "grayscale" in the environment parameters, and then used the `ResizeObservation` wrapper to give the frames the dimension 84 x 84. I then used `FrameStackObservation` wrapper to stack consecutive frames in groups of 4 (resulting in dimensions 4 x 84 x 84).

Observations also had to be converted to PyTorch Tensor objects to match the input dimensions of the CNN. I did this by writing a helper function `transform_obs(obs)`, that converts the input to a Tensor object and permutes its dimensions to those expected by the CNN (width, height, channels).

CNN Architecture

Convolution Layers:

Slides a filter called a kernel over the image in order to extract features, and calculate the dot product between the filter weights and the area of the image a portion of the grid is overlaid on. The model we are using contains two 2D convolution layers as its first two layers, the first of which takes in an 84 x 84 x 4 stack (or multiple stacks) of preprocessed images.

Fully-Connected Layers:

Fully-connected layers have incoming inputs connected to every neuron in the layer. The output dimensions are equal to the number of nodes in the layer.

These layers require convolution layer output dimensions to be flattened before processing them. Therefore, we apply a `flatten` function to the output from Conv Layer 2 before passing data to the first fully-connected layer.

Layer	Input	Output	Number of Filters	Filter Dimensions	Stride (number of pixels that filter window “slides” after operation)	Activation Function
Conv Layer 1	4 channels	16 channels	16	8 x 8	4	ReLU
Conv Layer 2	16 channels	32 channels	32	4 x 4	2	ReLU
Apply Flatten function to Conv Layer 2 output						
FC Layer 1	2592 features	256 features	NA	NA	NA	NA
FC Layer 2	256 features	18 features	NA	NA	NA	NA

Testing and Troubleshooting:

Setting up the CNN took a while, as the layers require very specific input dimensions that need to be calculated beforehand and I kept getting errors stating that my input dimensions didn't match a layer, even when I thought they did. Thankfully, I was able to find a resource on Baeldung that outlined the formulas required for this:

W = image width, H = image height, K = kernel dimensions, S = stride, P = padding

- $$W_{out} = \frac{W_{in} - K + 2P}{S} + 1$$
- $$H_{out} = \frac{H_{in} - K + 2P}{S} + 1$$

From Baeldung article “Calculate the Output Size of a Convolutional Layer”

It also took me a while to understand that an observation consisting of 4 grayscale images would mean the observation had 4 channels (grayscale images have 1 channel, so 4 * 1). That realization made the process of testing the CNN a lot easier. I also tried passing singular stacks of images to my CNN to make sure that the output was in the correct shape.

I had some data type issues when it came to converting my observation from the environment to a Tensor object using the standard torch library. A StackOverflow post suggested using torchvision's method for this instead, and it worked well. I spent a lot of time using print statements to test the result of transforming my observations.

During testing, I also realized that the way I handled epsilon decay also required a lot of troubleshooting. Initially, I had set epsilon to decay exponentially after every step. However, I eventually realized that exponential decay at every step would result in the agent not getting much time to explore the environment, with the agent resorting to exploitation before much learning had occurred. I then changed the decay to be linear, and to decrease after set intervals.

Experiments

The Atari handbook for this game makes it clear that it's pretty reward-based. Your goal is simply to maximize your reward. For that reason, my experiments focused on getting the highest possible scores I could, with a limited number of game episodes.

For each experiment, I set the number of episodes to 100. This is rather low, but as you get 3 lives with each episode, the agent technically gets 300 training opportunities. To measure the reward over time, I considered 10 episodes to be an epoch, and recorded the average total reward per epoch. Epsilon was set to 0.9 at the beginning of each episode, and was reduced by a fixed value of epsilon decay every 2 epochs. I felt that this would give the agent the opportunity to explore, and then exploit what it had learned about the environment towards the end of the experiment.

Short Clips of Experiments: <https://www.youtube.com/playlist?list=PLCjBvXr0uySM>

Experiment 1:

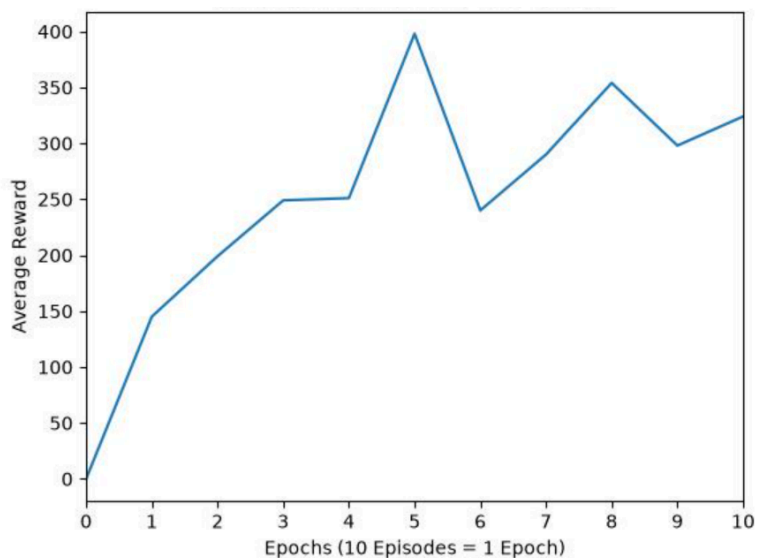
Hyperparameter	Value
gamma	0.7
alpha	0.1
epsilon	0.9

epsilon_decay	0.2
batch_size	32
num_episodes	100

This experiment was pretty interesting. The agent showed some variety in movement and seemed to be learning to avoid the aliens. Frustratingly, at a certain point of training, I realized it was moving towards aliens, often while it still had flamethrower fuel. The payoff for destroying even 1 alien is much higher than crushing one egg, and there are no negative rewards for being caught by one. I initially thought that the agent prioritized this large reward over the smaller ones from crushing eggs, but I am not sure if it destroyed an alien prior to this behaviour (which it needs to do to be aware of the reward).

It's more likely that the agent decided that dying and moving to the next human was the optimal strategy for quickly gathering points. Judging by the graph, that would work quite well if the sole purpose of the game was to use the three humans to maximize rewards across an episode.

Average epoch reward across 100 episodes:



Experiment 2:

HINTS FROM DALLAS NORTH...

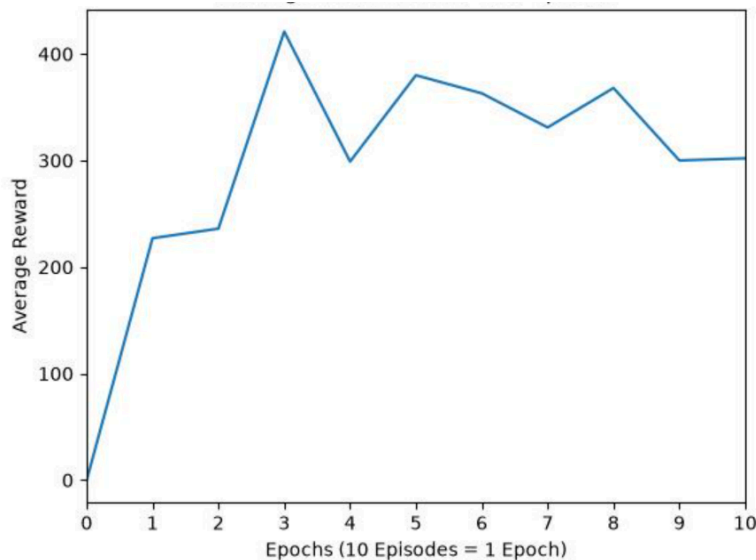
A good playing strategy is to crush all of the Eggs in one area at a time, keeping within easy reach of a Pulsar. The best way to destroy Aliens is to sit near a Pulsar until the Aliens are almost upon you. Then grab that Pulsar and go get 'em !

This tip is featured in the Atari handbook for the game. It's interesting to see humans being encouraged to prioritize closer rewards. The agent might benefit from trying that too, when it comes to crushing nearby eggs at least. A lower gamma value might help, as it would heavily discount future rewards. I also decided to try decaying epsilon faster, as I felt the agent spent too long on exploration in the previous experiment, and that random actions late into training disrupted its progress.

Hyperparameter	Value
gamma	0.4
alpha	0.1
epsilon	0.9
epsilon_decay	0.4
batch_size	32
num_episodes	100

I actually saw the agent successfully using the flamethrower on aliens this time, and it even destroyed one with a Pulsar rifle. An alien also followed me through the Hyperwarp Passage, something else I had never seen. However, the agent still seems to think that running into the aliens is the best strategy in the specific situation where it is being chased and has no flamethrower fuel left. This means that dying is considered the optimal choice in that state only, because the agent does not actively search for aliens. I did notice the agent being more efficient about crushing nearby eggs, which means decreasing gamma was a good move.

Average epoch reward across 100 episodes:



Instead of trying to make the agent reward focused, I think I should have tried making it more focused on what would actually complete the level, which is crushing eggs as a primary focus, and picking up the Pulsar rifle as a secondary focus. A higher reward for each of these actions might help, with a negative reward for being caught by an alien so that the agent actually runs away from them.

Conclusion

I definitely spent too much time planning this project, and had to rush my experiments and code. There are many ways in which this agent can be improved during training that I didn't get to try:

1. Introducing negative rewards: There are no negative rewards in this game, so it would be interesting to use the reward wrappers to change this. For example, being caught by an alien could result in a point deduction that would make the agent more likely to move away. Increased rewards for eggs could also be introduced to encourage the agent to prioritize crushing them.
2. I used a single CNN in this project, following the Deepmind paper. However, I underestimated the benefits of using two models in the way the PyTorch DQN tutorial did, with one policy agent and one training agent.
3. Training the agent for a lot longer would be ideal, maybe using the GPU and frame skipping.

4. Alpha decay would have been tricky but interesting to implement, especially with more episodes and a more goal-focused agent.

Something that concerned me was that when I tried to save my model's state dictionary from Experiment 2 and load it into a new Alien environment to test out the agent, it did not work in the way that it did during training at all. The videos uploaded are from the end of training rounds. There are many reasons due to which this could be happening, but it will take a while to pinpoint the issue.

Overall, there's still a lot to be done to improve this agent in terms of playing Alien, and I am looking forward to implementing these ideas soon.

References:

Playing Atari with Deep Reinforcement Learning (Mnih et al., 2013):

<https://arxiv.org/pdf/1312.5602>

Video- [Classic] Playing Atari with Deep Reinforcement Learning (Paper Explained):

<https://www.youtube.com/watch?v=rFwQDDbYTm4>

PyTorch DQN Tutorial (Paszke and Towers, 2017):

https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Gymnasium Atari Environments (Alien):

<https://ale.farama.org/environments/alien/>

Alien Atari Manual:

https://atariage.com/manual_html_page.php?SoftwareID=815

Stanford CNN Cheatsheet:

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/>

Calculate the Output Size of a Convolutional Layer:

<https://www.baeldung.com/cs/convolutional-layer-size>